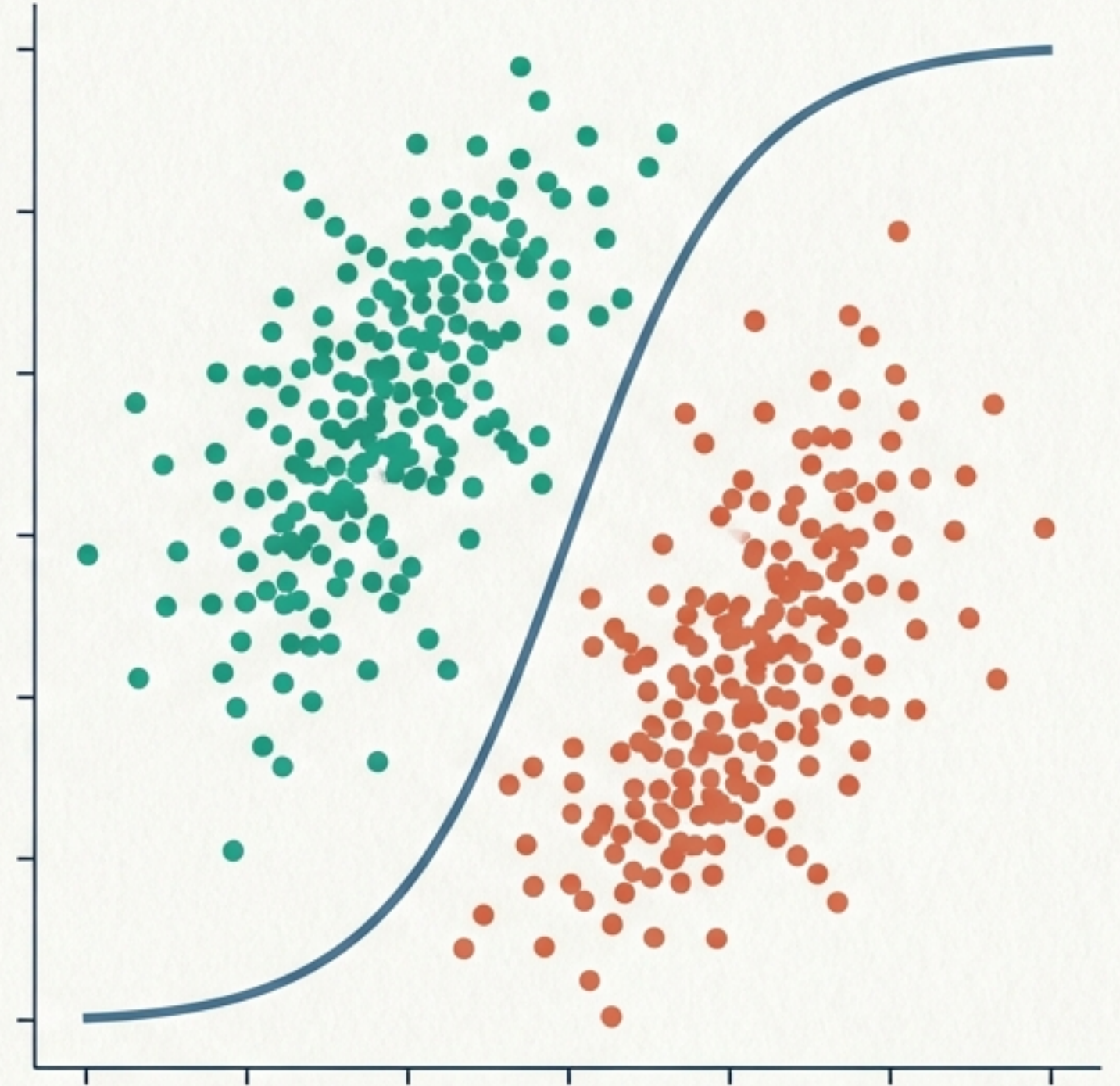


UT6: Modelos de Clasificación Supervisada

Teoría, Algoritmos y Evaluación de Sistemas de Aprendizaje Automático

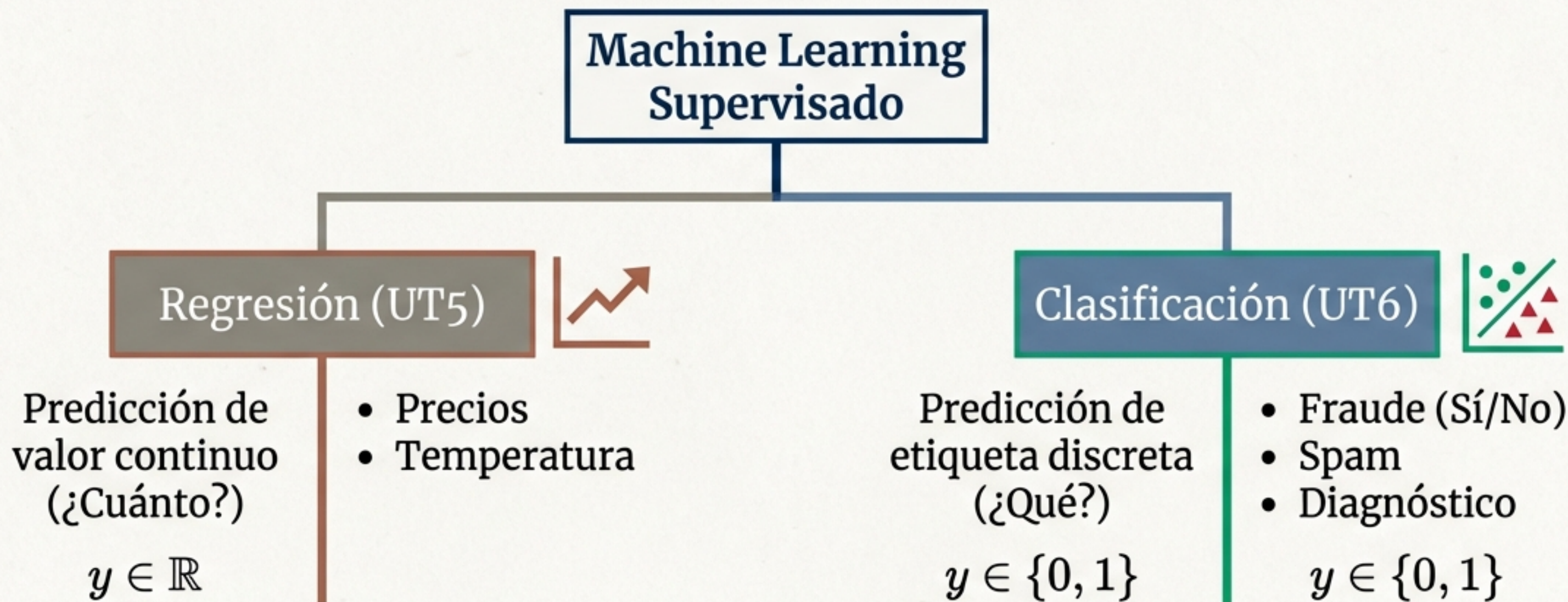
Una guía integral de estudio ("La Biblia de la Unidad").
Desde la matemática de las distancias hasta la optimización de hiperparámetros.



Experto Docente: Juan Alcaraz | Inteligencia Artificial y Big Data

El Mapa del Aprendizaje Supervisado

De la Regresión a la Clasificación



Insight: La diferencia fundamental reside en la variable dependiente. Mientras la regresión estima infinitas posibilidades, la clasificación asigna etiquetas discretas.

1. Regresión Logística: Probabilidad y Fronteras Lineales

Aunque su nombre indica “regresión”, se usa para **clasificación binaria**.

Transforma una ecuación lineal en una **probabilidad (0 a 1)** usando la **Función Sigmoide**.

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

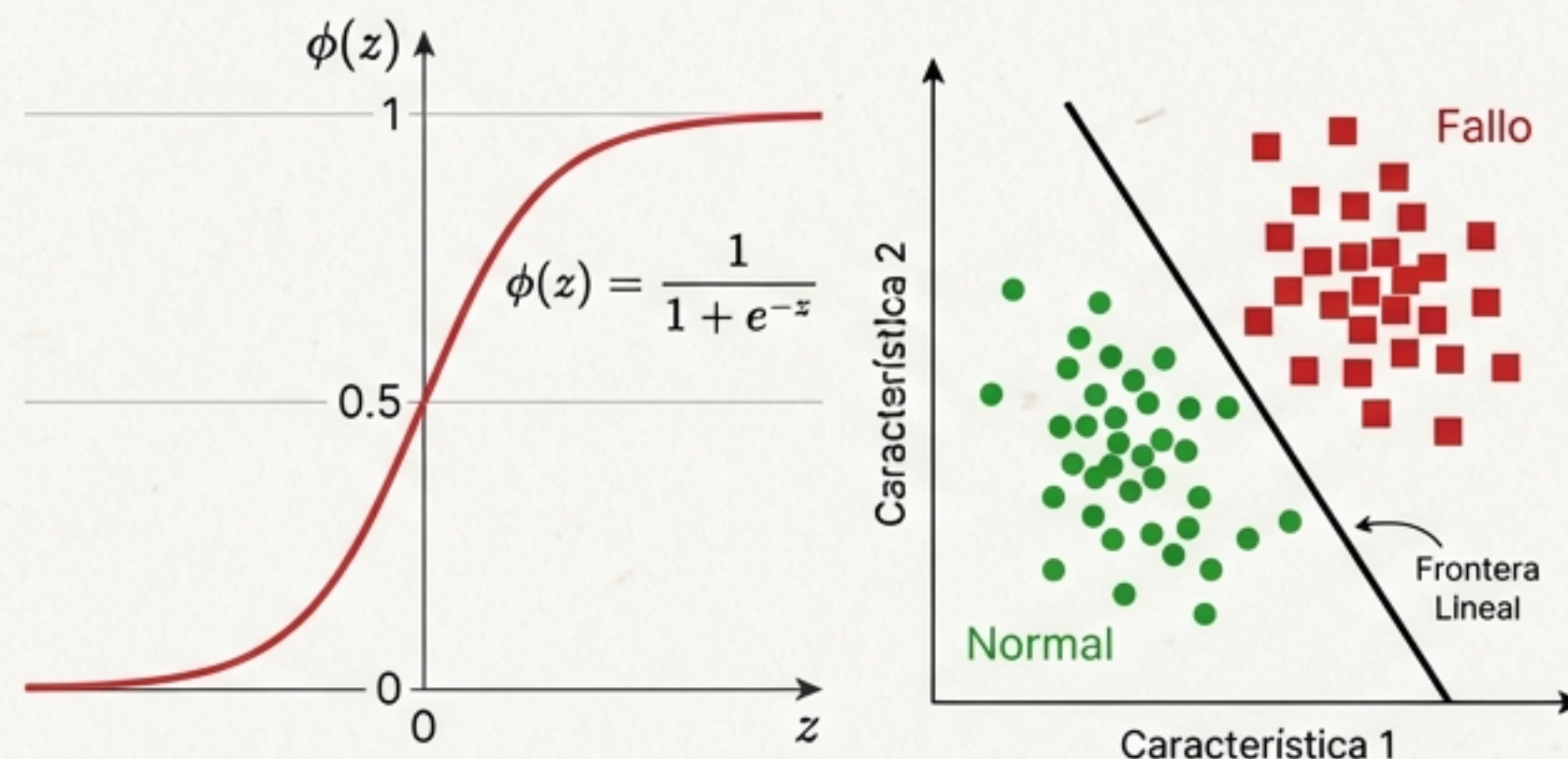
➡ Si $P > 0.5 \rightarrow$ Clase 1 (Fallo) ⚠

➡ Si $P \leq 0.5 \rightarrow$ Clase 0 (Normal) ✅

✅ Rápido y explicable.

❌ Limitado a fronteras lineales.

Visualización y Código



```
from sklearn.linear_model import LogisticRegression

# C controla la regularización (Inverso: C alto = menos penalización)
model = LogisticRegression(C=1.0, random_state=42)
```


2. K-Nearest Neighbors (KNN): La Lógica de la Vecindad

Concepto: Algoritmo “Lazy Learner”. No entrena un modelo matemático; memoriza la geografía de los datos. Clasifica por “votación democrática” de sus K vecinos.

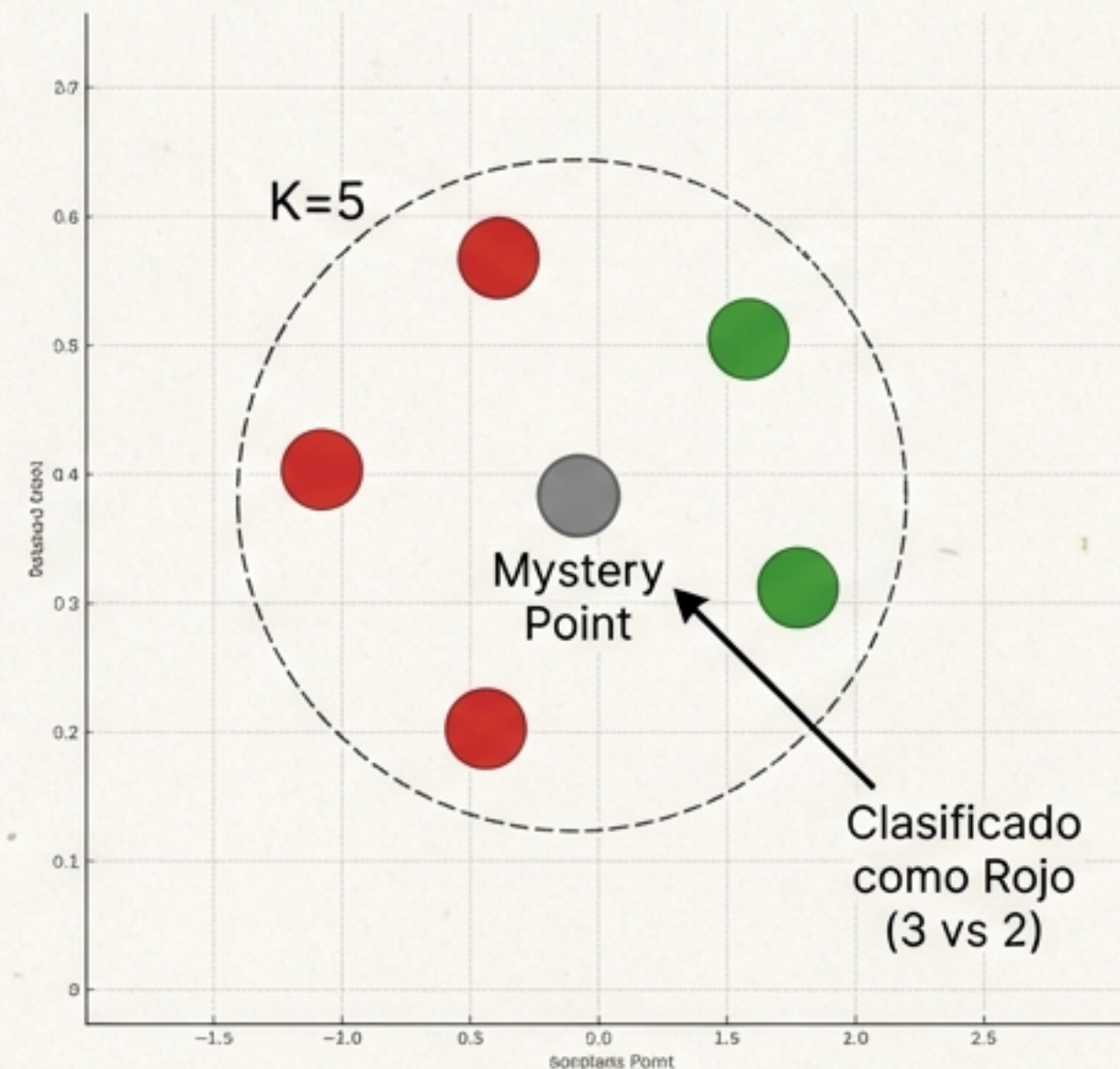
Analogía: “Dime con quién andas y te diré quién eres.”

Parameter K :

K bajo (1): Sensible al ruido (Overfitting).

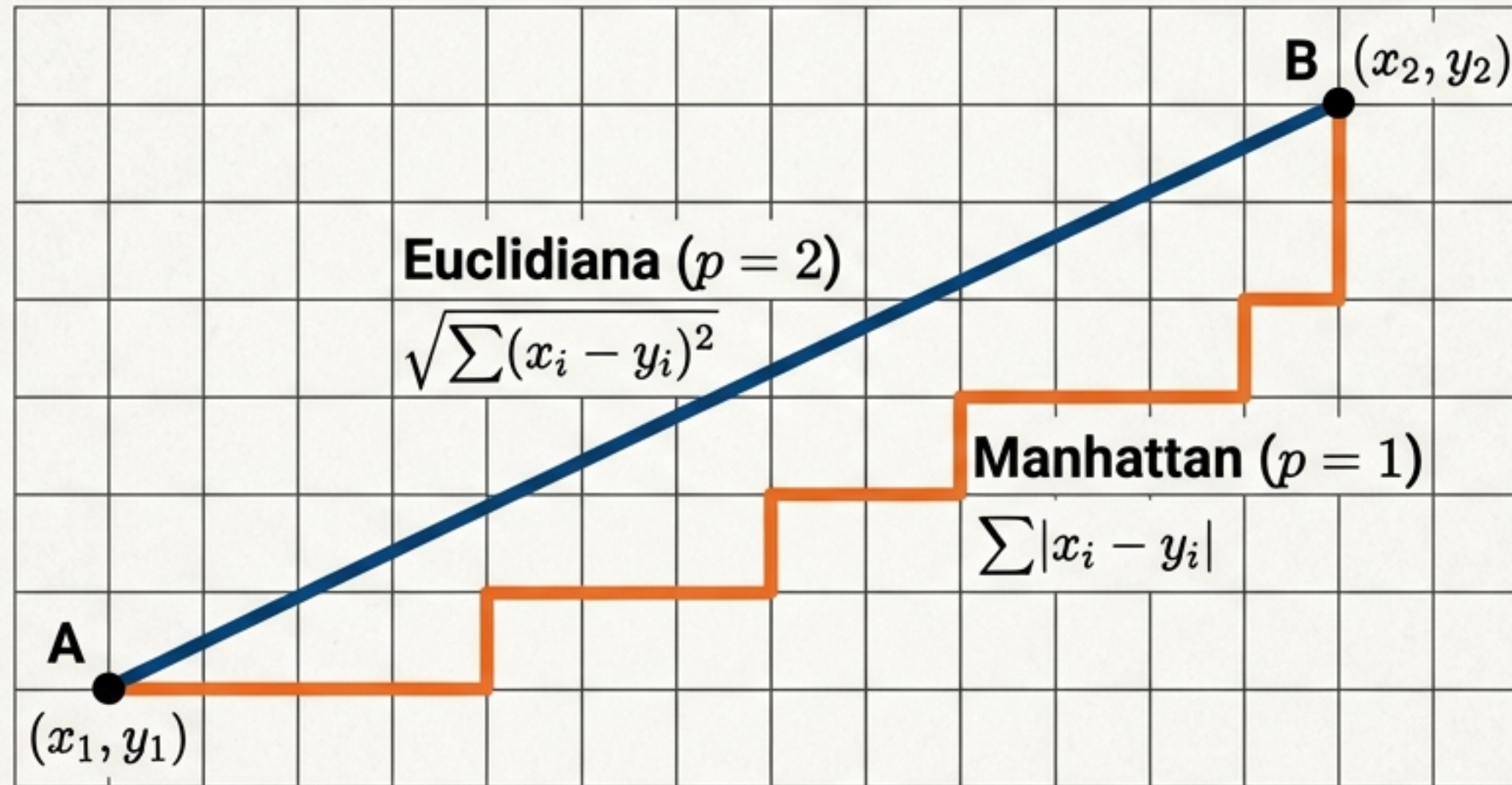
K alto: Pierde detalle, suaviza demasiado.

```
from sklearn.neighbors import KNeighborsClassifier
# n_neighbors define el número de votantes
model = KNeighborsClassifier(n_neighbors=5, metric='minkowski')
```



Fundamentos Matemáticos: Métricas de Distancia

Como medimos la similitud en Machine Learning (Minkowski)

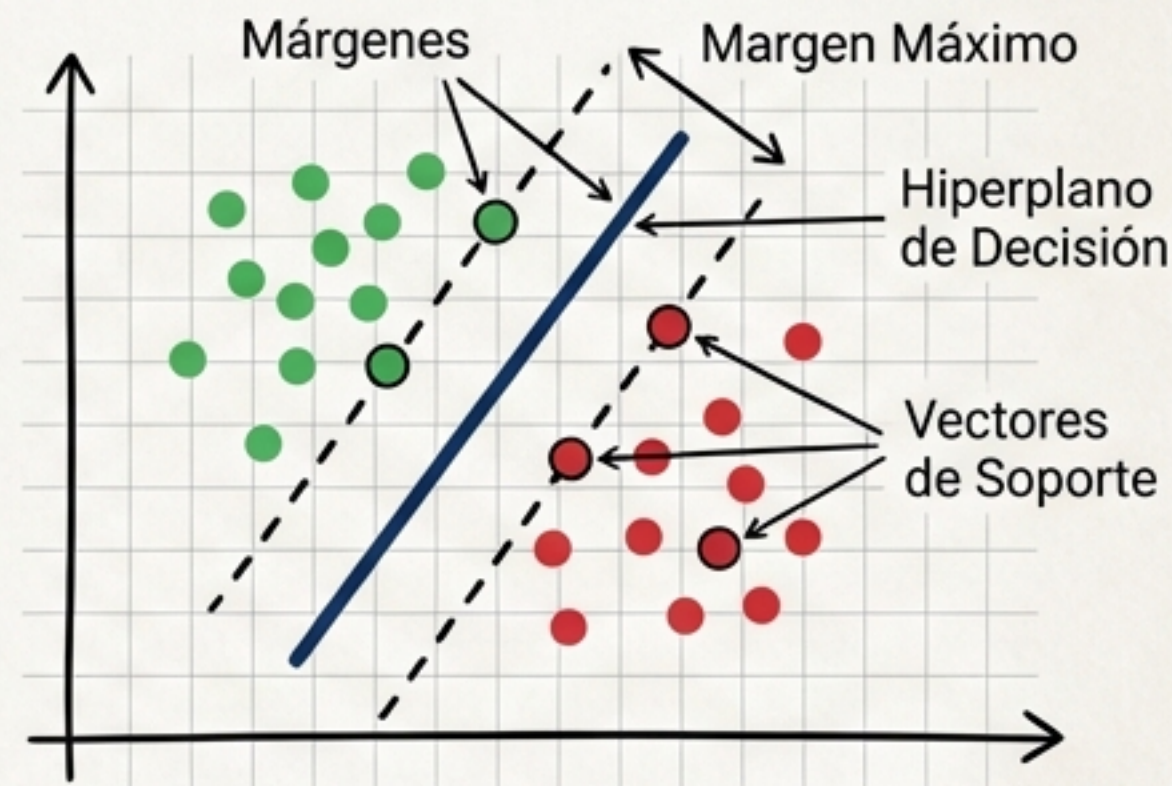


Euclidiana: Vuelo de pájaro. La línea recta más corta. Fundamental para variables continuas físicas. (Default en sklearn).

Manhattan: Taxista en la ciudad. Suma de bloques. Movimiento en ángulos rectos.

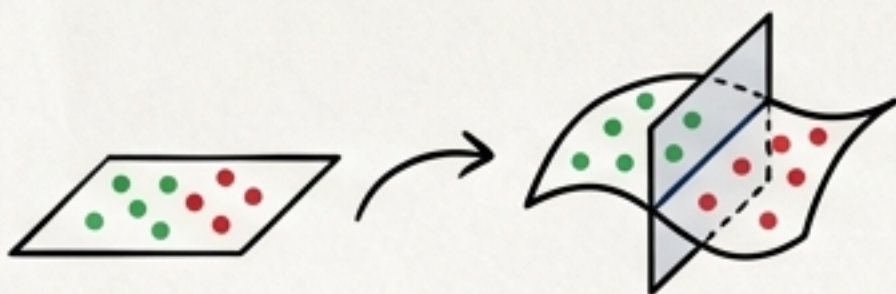
3. Support Vector Machines (SVM): El Margen Máximo

Concepto: Busca el hiperplano que crea la “calle” más ancha posible entre las clases.



El Truco del Kernel

Cuando los datos no son separables en 2D, SVM los proyecta a 3D para separarlos con un plano.



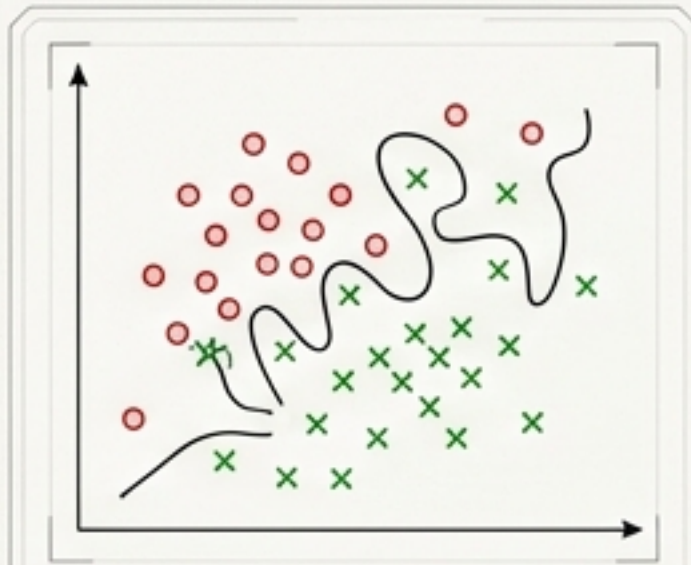
Código

```
from sklearn.svm import SVC

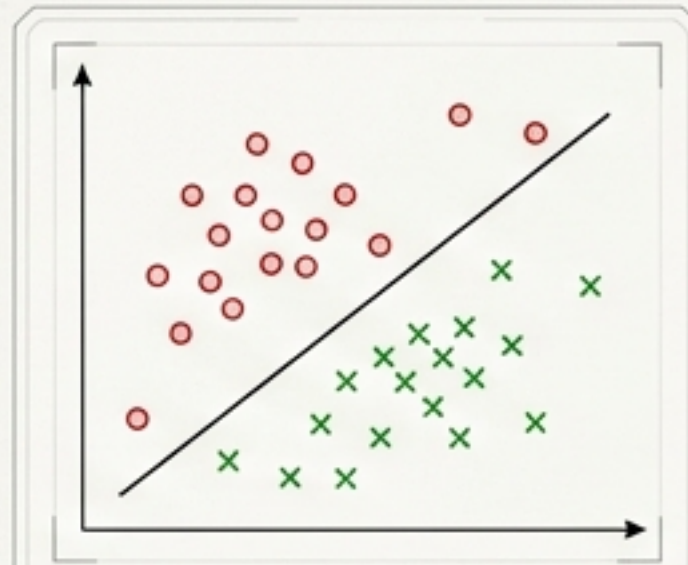
# Kernel RBF para fronteras curvas
model = SVC(kernel='rbf', C=1.0, gamma='scale')
```


Ajuste Fino: Regularización (C) y Alcance (Gamma)

Parámetro C (Penalización)



C Alto: Estricto
(Riesgo Overfitting)



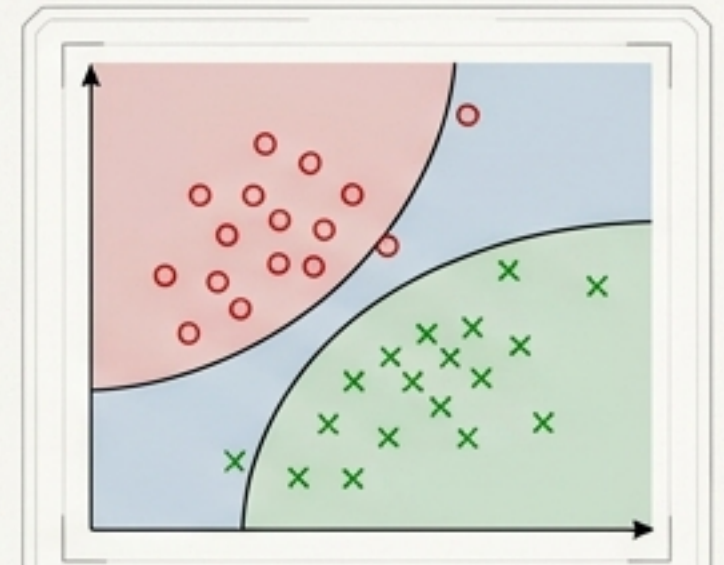
C Bajo: Permisivo
(Generalización)

Controla el precio de equivocarse. C alto penaliza errores; C bajo busca simplicidad.

Parámetro Gamma (Alcance – solo RBF)



Gamma Alto:
Influencia Local



Gamma Bajo:
Influencia Global

Controla la distancia de influencia de cada ejemplo. Gamma alto genera 'islotes'; Gamma bajo busca fronteras suaves.

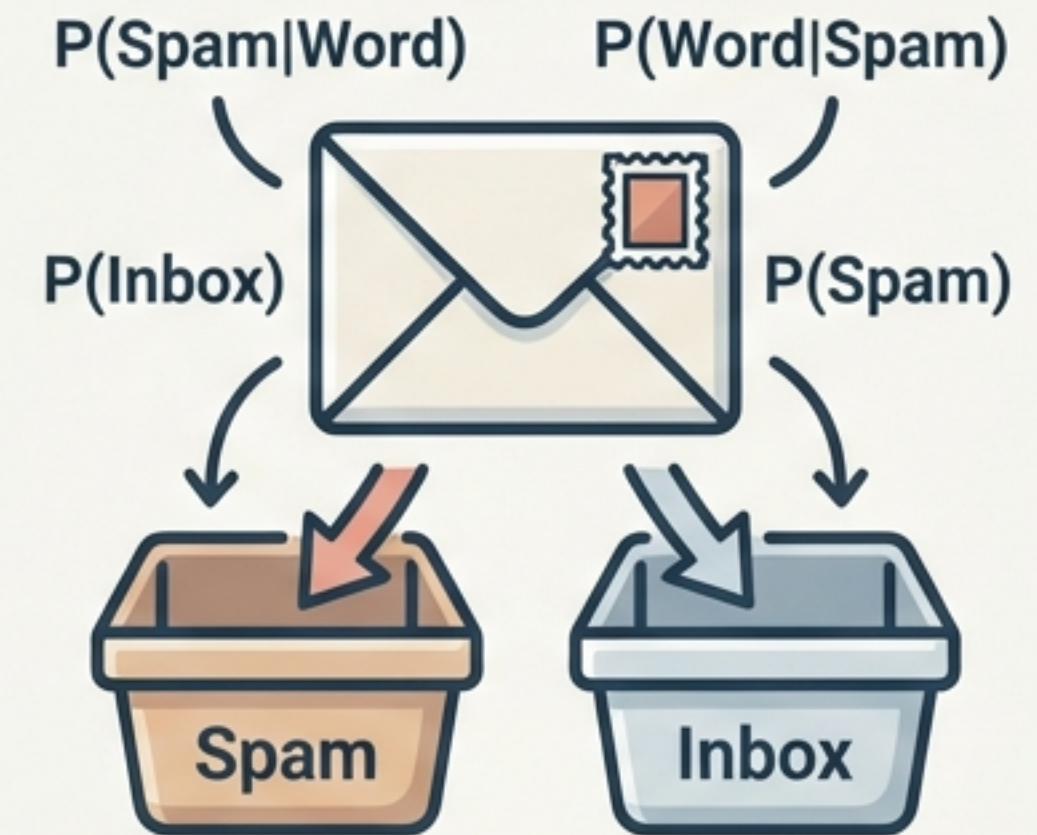
4. Naive Bayes: Probabilidad e Independencia

- **Concepto Teórico:** Basado en el Teorema de Bayes. Calcula la probabilidad $P(A|B)$.
- **Asunción "Ingenua" (Naïve):** Asume que todas las variables son independientes entre sí (ej. Tos y Fiebre no están relacionadas). Simplifica el cálculo computacional.

Fórmula del Teorema de Bayes:

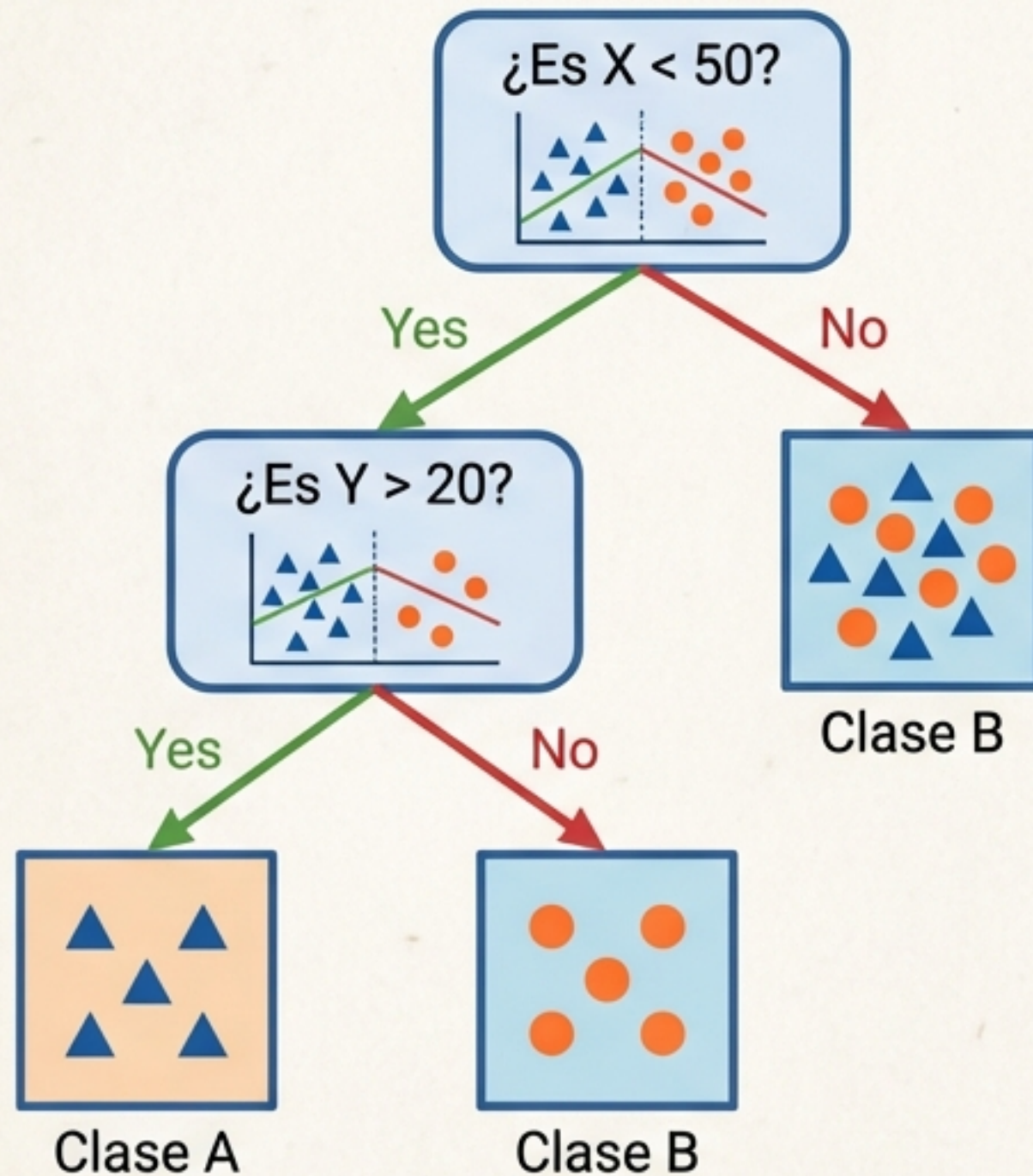
$$P(A|X) = \frac{P(X|A) \cdot P(A)}{P(X)}$$

🏆 Estándar para Clasificación de Texto (Spam vs Ham).



```
from sklearn.naive_bayes import  
GaussianNB  
model = GaussianNB()
```


5. Árboles de Decisión: Reglas Jerárquicas



Concepto

Divide el espacio mediante preguntas jerárquicas ('White Box').

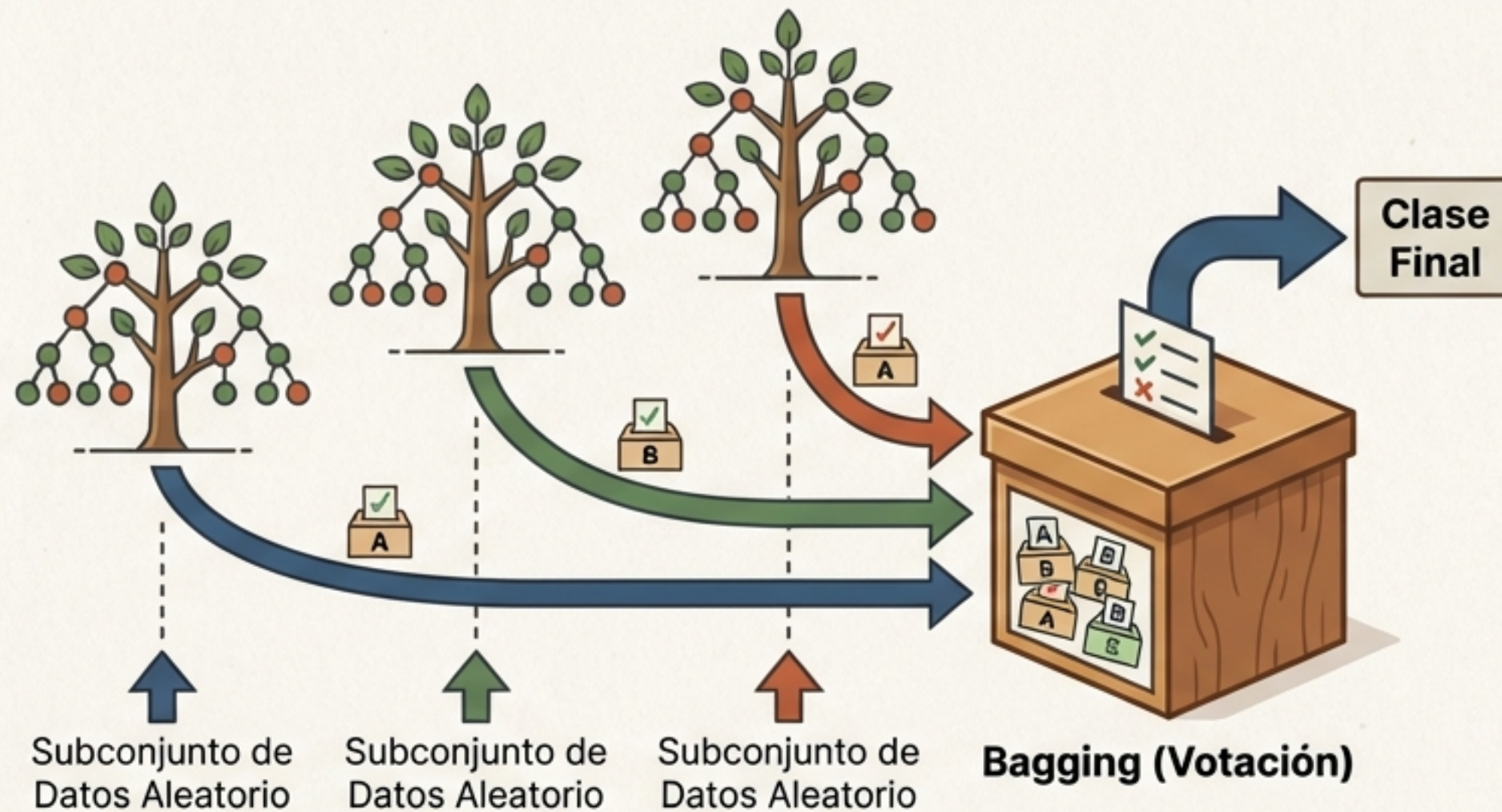
Métricas de Corte

- **Entropía:** Medida del desorden (0 = Pureza, 1 = Caos).
- **Ganancia de Información:** Cuánto reduce el desorden una pregunta.

Code

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(criterion='entropy')
```


6. Random Forest: La Sabiduría del Grupo



Concepto: Un conjunto (Ensemble) de múltiples árboles. Cada uno entrena con un subconjunto aleatorio de datos.

Ventaja: Soluciona el Overfitting de los árboles individuales. Robusto y preciso.

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100)
```


Guía de Selección Estratégica

Algoritmo	Pros	Contras
Regresión Logística	Simple, probabilidades	Solo lineal
KNN	Intuitivo, sin entreno	Lento, sensible a escalas
SVM	Alta precisión, Kernels	Caja negra, requiere escalado
Naive Bayes	Muy rápido (texto)	Asume independencia
Random Forest	Robusto, preciso	Costoso, pesado

Teacher's Tip: Comienza con LogReg/Trees (Baseline). Escala a Random Forest/SVM para rendimiento.

Evaluación I: La Matriz de Confusión

	Predicción: SANO (0)	Predicción: FALLO (1)
Real: SANO (0)	True Negative (TN) Acierto. Sano y predicho Sano.	False Positive (FP) Falsa Alarma. Predicho Fallo, pero Sano.
Real: FALLO (1)	⚠ False Negative (FN) ¡PELIGRO! Predicho Sano, pero Falló. Error más grave.	True Positive (TP) Acierto. Fallo detectado.

Evaluación II: Métricas de Rendimiento

Accuracy

Fórmula:



$$(TP+TN)/Total$$

Tasa general de aciertos. Cuidado con datos desbalanceados.

Precision

Fórmula:



$$\frac{TP}{(TP+FP)}$$

Calidad de la alarma. ¿Cuántos avisos fueron reales? (Clave en Spam).

Recall

Fórmula:



$$\frac{TP}{(TP+FN)}$$

Capacidad de detección. ¿Cuántos fallos reales atrapé? (Clave en Medicina).

F1-Score

Fórmula:

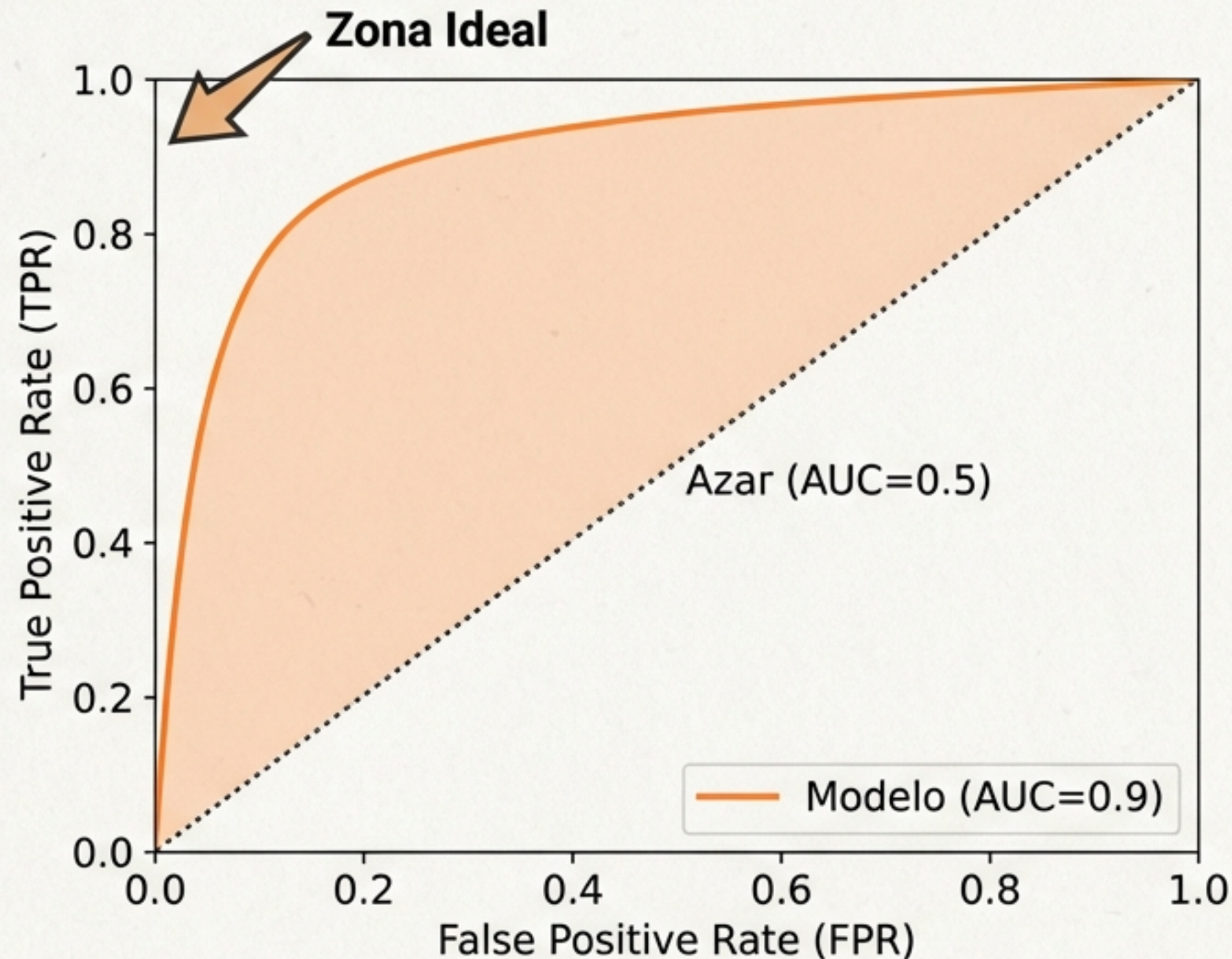


Media Armónica

Equilibrio entre Precision y Recall.

```
print(classification_report(y_test, y_pred))
```


Evaluación III: Curva ROC y AUC



AUC (Area Under Curve)

Nota final del modelo (0 a 1).

1.0 = Perfecto | 0.5 = Aleatorio.

Optimización: Grid Search y Validación Cruzada

K-Fold: Entrena K veces rotando el set de prueba para evitar la suerte.



Grid Search: Fuerza bruta para encontrar los mejores Hiperparámetros.

```
params = {'n_estimators': [10, 50, 100], 'criterion': ['gini', 'entropy']}  
grid = GridSearchCV(RandomForestClassifier(), params, cv=5)  
grid.fit(X, y)
```